YZ 203 - Week 4: Lab / Practice Session Notes

Lab Objectives

This lab starts with a still picture and ends with a first drawn version of the egg game. Work through the exercises in order; each one adds one idea.

By the end of this session, the student will be able to:

- Write a `JPanel` subclass and override `paintComponent(Graphics g)`
- Draw rectangles, ovals, lines, polygons, arcs and text, with colors
- Explain why `super.paintComponent(g)` is the first line and what `repaint()` does
- Animate a drawing with a `Timer` that updates numbers and calls `repaint()`
- Explain how the Timer was used in Week 3 (move a component) and how it is used now (update state, request a repaint)
- Put drawing code inside a small class (`Egg.draw(Graphics g)`) and use it from a panel

Level	Exercises
Easy (static drawing)	1. First paint, 2. Traffic light, 3. House
Easy to medium	4. Smiley, 5. Scoreboard text
Medium (animation)	6. Bouncing ball, 7. Falling oval egg
Hard (towards the game)	8. Egg class, 9. Basket and egg, 10. Falling Egg v1, 11. Challenge: three eggs

Duration: about 2 class hours. **Required materials:** JDK and NetBeans from Week 1.

Every exercise starts with `import javax.swing.*;` and `import java.awt.*;`. With these two imports, `Timer` means `javax.swing.Timer`.

Exercise 1 (Easy): Your First Drawing

Goal: Write your own panel class and draw one shape on it. No Timer yet.

```
import javax.swing.*;
import java.awt.*;

class GamePanel extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
```

```
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillRect(40, 30, 100, 60);
    }
}

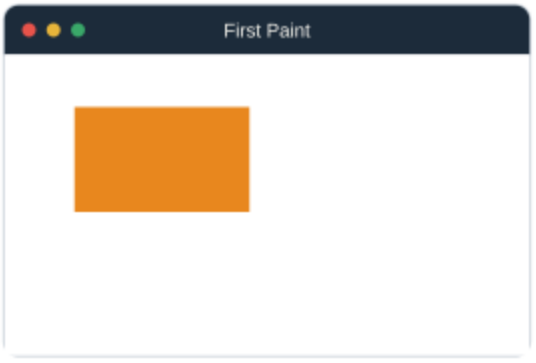
public class FirstPaint {
    public static void main(String[] args) {
        JFrame f = new JFrame("First Paint");
        f.add(new GamePanel());
        f.setSize(300, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Both classes go in one file named `FirstPaint.java`. Only the class with `main` is `public`.

Step by step:

1. `GamePanel extends JPanel`: our class is a panel with our own painting added.
2. `paintComponent` is overridden. `@Override` makes the compiler check that the name and signature are right.
3. `super.paintComponent(g)` clears the panel first. `setColor` picks the pen color, `fillRect(x, y, width, height)` draws a filled rectangle whose top-left corner is at (40, 30).
4. We never call `paintComponent` ourselves: Swing calls it when the window is shown.

Expected output:



Try it: change `fillRect` to `drawRect` and see the outline only. Then misspell the method as `paintcomponent` with a small c and read the compiler message that `@Override` produces.

Exercise 2 (Easy): A Traffic Light

Goal: Use several colors and shapes in the right order.

```
import javax.swing.*;
import java.awt.*;

class LightPanel extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.DARK_GRAY);
        g.fillRect(60, 20, 60, 160);
        g.setColor(Color.RED);
        g.fillOval(70, 30, 40, 40);
        g.setColor(Color.YELLOW);
        g.fillOval(70, 80, 40, 40);
        g.setColor(Color.GREEN);
        g.fillOval(70, 130, 40, 40);
    }
}

public class TrafficLight {
    public static void main(String[] args) {
        JFrame f = new JFrame("Traffic Light");
        f.add(new LightPanel());
        f.setSize(190, 240);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. The dark body is drawn first, the lights on top of it. Later shapes paint over earlier ones.
2. `fillOval(70, 30, 40, 40)` fits a circle into a 40 x 40 box whose top-left corner is (70, 30).
3. `setColor` stays active until you change it, so each light needs its own `setColor` before its `fillOval`.

Expected output:



Try it: move the `fillRect` line to the end of the method. The body now hides all three lights. This is why the drawing order matters.

Exercise 3 (Easy): A House

Goal: Build a picture from simple shapes, including a polygon.

```
import javax.swing.*;
import java.awt.*;

class HousePanel extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(new Color(200, 160, 120));
        g.fillRect(60, 100, 120, 80);
        g.setColor(Color.RED);
        int[] xs = {50, 120, 190};
        int[] ys = {100, 50, 100};
        g.fillPolygon(xs, ys, 3);
        g.setColor(Color.DARK_GRAY);
        g.fillRect(105, 130, 30, 50);
    }
}

public class HouseDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("House");
        f.add(new HousePanel());
        f.setSize(250, 250);
```

```
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. `new Color(200, 160, 120)` mixes red, green and blue (0 to 255 each) into a wall color.
2. A polygon is given as two arrays, one of x values and one of y values, plus the number of points. Here the three points form the roof triangle.
3. The door is the last shape, drawn on top of the wall.

Expected output:



Try it: add a yellow circle for a sun in the top-left corner and a window (a small light-blue rectangle) on the wall.

Exercise 4 (Easy to Medium): A Smiley Face

Goal: Draw a curve with `drawArc`, and combine a filled shape with an outline.

```
import javax.swing.*;
import java.awt.*;

class SmileyPanel extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.YELLOW);
        g.fillOval(50, 30, 100, 100);
        g.setColor(Color.BLACK);
        g.drawOval(50, 30, 100, 100);
        g.fillOval(80, 60, 10, 14);
        g.fillOval(110, 60, 10, 14);
```

```
        g.drawArc(75, 70, 50, 40, 200, 140);
    }
}

public class Smiley {
    public static void main(String[] args) {
        JFrame f = new JFrame("Smiley");
        f.add(new SmileyPanel());
        f.setSize(230, 190);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. `fillOval` colors the face yellow; `drawOval` with the same numbers adds the black outline.
2. The eyes are two small filled ovals.
3. `drawArc(x, y, w, h, startAngle, sweep)` draws part of an oval. Angle 0 is at 3 o'clock and positive angles go counter-clockwise, so start 200 with a sweep of 140 is the lower curve: a smile.

Expected output:



Try it: change the sweep to `-140` or move the start angle to make the face sad.

Exercise 5 (Easy to Medium): A Scoreboard with drawString

Goal: Draw text, and keep the numbers in private fields.

```
import javax.swing.*;
import java.awt.*;

class ScorePanel extends JPanel {
    private int score = 120, lives = 3;
```

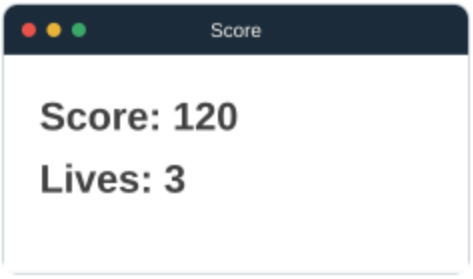
```
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.DARK_GRAY);
        g.setFont(new Font("Arial", Font.BOLD, 22));
        g.drawString("Score: " + score, 20, 40);
        g.drawString("Lives: " + lives, 20, 75);
    }
}

public class ScoreText {
    public static void main(String[] args) {
        JFrame f = new JFrame("Score");
        f.add(new ScorePanel());
        f.setSize(260, 150);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. `score` and `lives` are private fields. `paintComponent` only reads and displays them.
2. `drawString(text, x, y)`: the `y` value is the baseline, the bottom of the letters, not the top.
3. `setFont` changes the font for every later `drawString`.

Expected output:



Try it: add a method `void addScore(int points)` that increases `score` and calls `repaint()`. Call it from a JButton to see the text update.

Exercise 6 (Medium): A Bouncing Ball with Timer and repaint()

Goal: Animate a drawing. The Timer updates numbers and asks for a repaint; `paintComponent` does all the drawing.

```
import javax.swing.*;
import java.awt.*;

class BallPanel extends JPanel {
    private int x = 0, dx = 4;

    BallPanel() {
        new Timer(16, e -> {
            x += dx;
            if (x < 0 || x > getWidth() - 40) dx = -dx;
            repaint();
        }).start();
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillOval(x, 60, 40, 40);
    }
}

public class BallDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Bouncing Ball");
        f.add(new BallPanel());
        f.setSize(420, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. Compare with Week 3's `BounceDemo`: the Timer line, the `x += dx` and the wall rule are identical.
2. The only change is the last line of the listener: `box.setLocation(x, 50)` became `repaint()`. There is no component to move any more.
3. `x` and `dx` are private fields of the panel. The Timer is started in the constructor, so the panel animates itself.

Expected output:



Try it: delete `super.paintComponent(g);` and run again. The ball leaves a trail of old frames. Put the line back.

Exercise 7 (Medium): A Falling Oval Egg with a Random Column

Goal: Rebuild Week 3's falling egg as a real drawing. The egg restarts at the top in a random column.

```
import javax.swing.*;
import java.awt.*;

class FallPanel extends JPanel {
    private int x = 100, y = 0;
    private final java.util.Random rnd = new java.util.Random();

    FallPanel() {
        new Timer(16, e -> {
            y += 4;
            if (y > getHeight()) {
                y = 0;
                x = rnd.nextInt(getWidth() - 30);
            }
            repaint();
        }).start();
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillOval(x, y, 30, 40);
    }
}

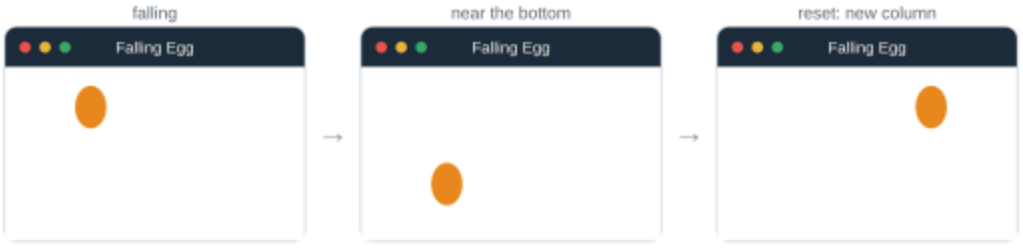
public class FallDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Falling Egg");
        f.add(new FallPanel());
        f.setSize(260, 340);
```

```
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. The reset rule (`y` past the bottom, new random `x`) is the same as in Week 3's `RandomFall` .
2. There is no `JLabel` , no `setBounds` and no `null` layout: the panel simply draws an oval at (`x` , `y`).
3. The `Random` is created once, as a field. Never create it inside `paintComponent` , which runs about 60 times a second.

Expected output:



Try it: resize the window while the egg falls. Notice that `getWidth()` and `getHeight()` follow the new size, so the egg always falls the whole height.

Exercise 8 (Medium to Hard): The Egg Draws Itself

Goal: Move the egg's state and its drawing into an `Egg` class. The panel only asks the egg to fall and to draw.

```
import javax.swing.*;
import java.awt.*;

class Egg {
    private int x, y;
    private final int speed;

    Egg(int x, int speed) {
        this.x = x;
        this.speed = speed;
    }

    // returns true when the egg wrapped back to the top
    boolean fall(int fieldHeight) {
        y += speed;
```

```
        if (y > fieldHeight) { y = 0; return true; }
        return false;
    }

    void setX(int x) { this.x = x; }

    void draw(Graphics g) {
        g.setColor(Color.ORANGE);
        g.fillOval(x, y, 30, 40);
        g.setColor(Color.WHITE);
        g.fillOval(x + 7, y + 8, 8, 10);
    }
}

class EggPanel extends JPanel {
    private final Egg egg = new Egg(120, 4);

    EggPanel() {
        new Timer(16, e -> {
            egg.fall(getHeight());
            repaint();
        }).start();
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        egg.draw(g);
    }
}

public class EggClassDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Egg class");
        f.add(new EggPanel());
        f.setSize(260, 340);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. Compare with Week 3's `Egg`: it no longer holds a `JLabel`. It holds only numbers and knows how to draw them on any `Graphics`.

2. `x`, `y` and `speed` are `private`. The only ways to change them are `fall()` and `setX()`.
3. The panel does not know what an egg looks like. It calls `egg.draw(g)` and hands over its own pen (`g`).

Expected output:



Try it: change the egg's look inside `Egg.draw()` only, for example a bigger highlight or a second color. Notice that `EggPanel` needs no change at all.

Exercise 9 (Medium to Hard): A Basket and an Egg on One Panel

Goal: Draw two different objects on one panel. The basket is a half oval whose position comes from the panel height. There is no collision yet: that comes after the keyboard in Week 5.

```
import javax.swing.*;
import java.awt.*;

class Egg {
    private int x, y;
    private final int speed;

    Egg(int x, int speed) { this.x = x; this.speed = speed; }

    boolean fall(int fieldHeight) {
        y += speed;
        if (y > fieldHeight) { y = 0; return true; }
        return false;
    }

    void setX(int x) { this.x = x; }

    void draw(Graphics g) {
        g.setColor(Color.ORANGE);
        g.fillOval(x, y, 30, 40);
        g.setColor(Color.WHITE);
        g.fillOval(x + 7, y + 8, 8, 10);
```

```
    }
}

class CatchPanel extends JPanel {
    private final Egg egg = new Egg(120, 4);
    private final java.util.Random rnd = new java.util.Random();

    CatchPanel() {
        new Timer(16, e -> {
            if (egg.fall(getHeight())) egg.setX(rnd.nextInt(getWidth() -
30));
            repaint();
        }).start();
    }

    private void drawBasket(Graphics g) {
        int x = getWidth() / 2 - 45, y = getHeight() - 70;
        g.setColor(new Color(139, 90, 43));
        g.fillArc(x, y, 90, 60, 180, 180);
        g.setColor(new Color(100, 62, 28));
        g.drawArc(x, y, 90, 60, 180, 180);
        g.drawLine(x, y + 30, x + 90, y + 30);
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        egg.draw(g);
        drawBasket(g);
    }
}

public class BasketDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Basket");
        f.add(new CatchPanel());
        f.setSize(280, 340);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. `fillArc(x, y, w, h, 180, 180)` draws the lower half of an oval: start angle 180 (9 o′clock), sweep 180 counter-clockwise through 6 o′clock to 3 o′clock.

2. The basket's `y` is `getHeight() - 70`, so it stays at the bottom when the window is resized.
3. The basket is drawn after the egg, so when the egg passes the basket the brown shape covers part of it. Drawing order again.

Expected output:



Try it: swap the order of `egg.draw(g)` and `drawBasket(g)`. Now the egg is painted over the basket instead of behind it. Which order looks like "inside the basket"?

Exercise 10 (Hard): Falling Egg v1

Goal: Rebuild Week 3's Falling Egg v0 with real graphics. The field is our own `FieldPanel`; the window code only talks to its two public methods, `toggle()` and `setSpeed()`.

```
import javax.swing.*;
import java.awt.*;

class Egg {
    private int x, y;
    private final int speed;

    Egg(int x, int speed) { this.x = x; this.speed = speed; }

    boolean fall(int fieldHeight) {
        y += speed;
        if (y > fieldHeight) { y = 0; return true; }
        return false;
    }

    void setX(int x) { this.x = x; }

    void draw(Graphics g) {
        g.setColor(Color.ORANGE);
        g.fillOval(x, y, 30, 40);
        g.setColor(Color.WHITE);
        g.fillOval(x + 7, y + 8, 8, 10);
    }
}
```

```
class FieldPanel extends JPanel {
    private final Egg egg = new Egg(100, 4);
    private final java.util.Random rnd = new java.util.Random();
    private final Timer timer;
    private int missed = 0;

    FieldPanel() {
        setPreferredSize(new Dimension(400, 360));
        setBackground(new Color(222, 240, 255));
        timer = new Timer(16, e -> {
            if (egg.fall(getHeight())) {
                missed++;
                egg.setX(rnd.nextInt(getWidth() - 30));
            }
            repaint();
        });
        timer.start();
    }

    boolean toggle() {
        if (timer.isRunning()) timer.stop(); else timer.start();
        return timer.isRunning();
    }

    void setSpeed(int delay) { timer.setDelay(delay); }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        egg.draw(g);
        g.setColor(Color.DARK_GRAY);
        g.drawString("Missed: " + missed, 10, 20);
    }
}

public class FallingEggV1 {
    public static void main(String[] args) {
        JFrame f = new JFrame("Falling Egg v1");
        FieldPanel field = new FieldPanel();
        JButton toggle = new JButton("Pause");
        JSlider speed = new JSlider(5, 40, 16);
        JPanel bar = new JPanel();
        bar.add(toggle);
        bar.add(new JLabel("Delay:"));
        bar.add(speed);
```

```
        toggle.addActionListener(e ->
            toggle.setText(field.toggle() ? "Pause" : "Start"));
        speed.addChangeListener(e -> field.setSpeed(speed.getValue()));

        f.add(field);
        f.add(bar, BorderLayout.SOUTH);
        f.pack();
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Step by step:

1. `FieldPanel` is a `JPanel` (inheritance) that has an `Egg` and a `Timer` (composition). The `timer` field is `private`: nobody outside can stop it by accident.
2. The Timer listener only updates state (`fall`, `missed`, `setX`) and calls `repaint()`. `paintComponent` only reads that state and draws it.
3. `setPreferredSize` plus `pack()` gives the field exactly 400 x 360 pixels, with the button bar added below.
4. The button and the slider talk to the field through `toggle()` and `setSpeed()`. `main` never sees the Timer.

Expected output:



Try it: while the game is paused, drag the slider and press Start. Does the new speed apply? Explain using what you know about `setDelay()` from Week 3.

Exercise 11 (Challenge): Three Eggs, Three Speeds

Goal: Reuse the `Egg` class for several objects, held in an array, with one Timer and one `paintComponent`. The skeleton below compiles and runs, but the eggs stay at the top until you finish the two TODOs.

```
import javax.swing.*;
import java.awt.*;

class Egg {
    private int x, y;
    private final int speed;
    private final Color color;

    Egg(int x, int speed, Color color) {
        this.x = x;
        this.speed = speed;
        this.color = color;
    }

    // returns true when the egg wrapped back to the top
    boolean fall(int fieldHeight) {
        y += speed;
        if (y > fieldHeight) { y = 0; return true; }
        return false;
    }

    void draw(Graphics g) {
        g.setColor(color);
        g.fillOval(x, y, 30, 40);
    }
}

class ThreeEggsPanel extends JPanel {
    private final Egg[] eggs = {
        new Egg(40, 3, Color.ORANGE),
        new Egg(140, 5, new Color(200, 60, 60)),
        new Egg(240, 8, new Color(58, 123, 213))
    };
    private int missed = 0;

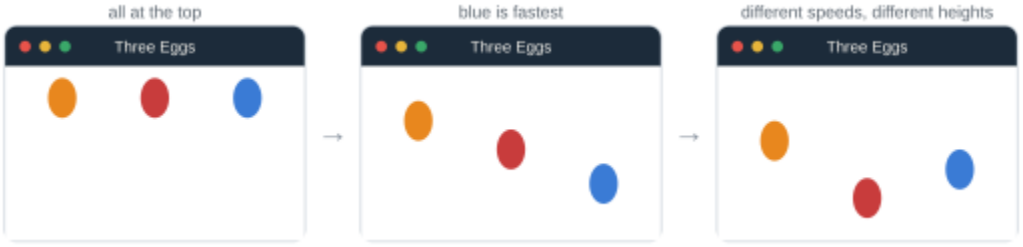
    ThreeEggsPanel() {
        setPreferredSize(new Dimension(300, 360));
        new Timer(16, e -> {
            // TODO 1: loop over eggs, call fall(getHeight()) on each,
            //         and add 1 to missed whenever fall returns true
            repaint();
        }).start();
    }

    @Override
    protected void paintComponent(Graphics g) {
```

```
        super.paintComponent(g);
        // TODO 2: draw every egg
        g.setColor(Color.DARK_GRAY);
        g.drawString("Missed: " + missed, 10, 20);
    }
}

public class ThreeEggs {
    public static void main(String[] args) {
        JFrame f = new JFrame("Three Eggs");
        f.add(new ThreeEggsPanel());
        f.pack();
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Expected output once both TODOs are done:



The blue egg is the fastest, the orange one the slowest, and the `Missed` text counts every egg that reaches the bottom.

Hints:

1. A loop over an array is `for (Egg egg : eggs) { ... }`. Do not name the loop variable `e`: the lambda already uses that name.
2. `fall()` returns a `boolean`, so `if (egg.fall(getHeight())) missed++;` works directly.
3. Do not call `fall()` twice on one egg in a tick, or it moves twice as far.
4. Think ahead: an array has a fixed size. What would you do for a game where new eggs appear over time? That question leads to `ArrayList` in a few weeks.

Behind the Scenes: The Timer in Week 3 vs Week 4, and the OOP Ideas

The Timer line is exactly the same in both weeks. What changes is the job its listener does.

	Week 3: colored `JLabel`	Week 4: our own `paintComponent`
Creating the Timer	`new Timer(16, e -> ...)`	the same
Last line of the listener	`box.setLocation(x, y);`	`repaint();`
Who draws the picture	the `JLabel`, a ready-made component	our `paintComponent(Graphics g)`
What we move	a component, by position	nothing: we only change numbers
Where `x` and `y` live	`static` fields in `main`	private fields of the panel or of an `Egg`
Layout	needs `null` layout	no layout tricks needed
100 eggs	100 `JLabel`s to manage	one panel that draws 100 shapes

The flow in Week 3 was: Timer, update `x`, `setLocation`, Swing repaints the label. The flow now is: Timer, update `x`, `repaint()`, Swing calls `paintComponent(g)`, we draw. **The Timer never draws. It updates numbers and asks for a repaint.**

OOP idea	Where you saw it
Inheritance	`GamePanel extends JPanel`: our panel is a `JPanel` plus our own painting (every exercise).
Overriding and polymorphism	`paintComponent` is replaced in our subclass. Swing holds a `JPanel` reference, yet at run time our version runs.
Callback	We never call `paintComponent`. Like `actionPerformed`, Swing calls it when needed.
Encapsulation	`Egg` hides `x`, `y`, `speed`; `FieldPanel` hides its `timer` behind `toggle()` and `setSpeed()` (Exercises 8, 10).
Composition	`FieldPanel` has an `Egg` and a `Timer`. It is not an `Egg` or a `Timer` (Exercise 10).
Separation of state and view	The listener updates, `paintComponent` draws, and neither does the other's job.

Homework / Research Questions

No answers are given here. Work them out and bring them to the next session.

1. Build it. Extend Exercise 10 with a `JCheckBox` labeled "Show trail" and a `JComboBox` of egg colors (Orange, Red, Blue). When the box is checked, `FieldPanel` must skip `super.paintComponent(g)`, so old frames stay on screen. The combo box must change the color the egg draws with. Which classes need a new public method, and which fields stay private? Describe in two sentences what you see when the trail is on and the window is then resized.

2. Investigate. `paintComponent` receives a `Graphics`, yet we cast it to `Graphics2D` to use `setStroke` and anti-aliasing. Look up `Graphics`, `Graphics2D` and `JComponent` in the Java documentation. Explain why the cast is allowed, what the real class of the object behind `g` is, and which two OOP ideas from this course make this possible. Then explain why Swing programmers override `paintComponent` and not `paint`: what else does `paint` do besides calling `paintComponent`?

Submission Checklist

- [ ] `FirstPaint.java`: your own `JPanel` subclass draws an orange rectangle
- [ ] `TrafficLight.java`: body first, three lights on top
- [ ] `HouseDemo.java`: wall, polygon roof and door
- [ ] `Smiley.java`: face, eyes and a smile made with `drawArc`
- [ ] `ScoreText.java`: score and lives drawn with `drawString` and a custom font
- [ ] `BallDemo.java`: the ball bounces; `super.paintComponent(g)` is the first line, and you tried removing it
- [ ] `FallDemo.java`: the oval egg restarts in a random column and follows window resizing
- [ ] `EggClassDemo.java`: `Egg` draws itself; the panel does not know what an egg looks like
- [ ] `BasketDemo.java`: basket at the bottom, egg behind or inside it depending on the drawing order
- [ ] `FallingEggV1.java`: Pause / Start, speed slider and the missed text all work
- [ ] `ThreeEggs.java`: both TODOs done, three speeds, one shared counter
- [ ] Can explain the difference between how the Timer was used in Week 3 and in Week 4
- [ ] Can explain what `repaint()` does and why we never call `paintComponent` ourselves
- [ ] Both homework questions answered

Next Week

Week 5 gives the basket a player: `KeyListener` and key bindings, `MouseListener` and `MouseMotionListener`, the keyboard focus problem, and the first collision test: did the egg land in the basket?